

Lecture Transcript — MSI Slide Automation Pipeline

Audience: Python-familiar developers/students

Format: Spoken walkthrough

The following is a word-for-word script. Pauses are marked with [PAUSE]. Emphasis words are in CAPS. Audience interaction points are in [brackets].

Opening

Alright, so what we're looking at here is a pipeline diagram. And I want you to think of the word "pipeline" very literally. Imagine water flowing through actual pipes — something goes in one end, it passes through several stages, and something different comes out the other end. That's exactly what's happening here.

What goes IN are two files — an Excel spreadsheet and a PowerPoint template. What comes OUT is a fully populated, multi-slide PowerPoint deck, ready to hand to a client. The question we're answering today is: **what happens in between?**

[PAUSE]

Let's read this diagram from top to bottom.

The Two Inputs

We have two completely separate inputs at the top — and they are intentionally separate. They serve different purposes.

Input One. The Excel file.

Now, most of you have worked with Excel. You know it stores data in rows and columns. A typical use case — column A is Item Number, column B is Name, column C is Price, and so on. Each row is one product. That's straightforward.

But here's where it gets interesting. This particular Excel file is doing something extra. It's not JUST storing data in cells. It also has product images — actual image files — floating INSIDE the spreadsheet. Not in a separate folder. Not in a separate upload. Embedded directly inside the .xlsx file's internal structure.

So when we look at Input One, we're actually pulling TWO things out of it. [PAUSE]

The first thing we pull out — using the `pandas` library — is the tabular data. All those rows and columns. Pandas reads the Excel file and gives us a DataFrame object. Think of a DataFrame as a Python-native version of a spreadsheet. Each row is accessible by index, each column is accessible by name. Easy to loop through, easy to query.

The arrow on the left says this tabular data goes toward building a **Text Replacements Dictionary**. We'll come back to what that means in a moment.

The SECOND thing we pull out — and this requires a different library, `openpyxl` — are the embedded images. `openpyxl` lets us go deeper into the Excel file's XML structure and access something called `ws._images`. That's an internal property that holds all image objects anchored to the worksheet.

Each image object has an anchor — basically coordinates telling us which ROW it belongs to. So we loop through all images, read that anchor row number, and store the image bytes against that row number in a dictionary. We call this the **image_map**. It looks like: row 2 has this image, row 3 has this image, row 5 has this image. A simple key-value store.

[PAUSE]

So just from ONE file — the Excel file — we've produced two separate data structures. A replacements dictionary being built, and an image_map. Hold that thought.

Input Two — The PowerPoint Template

Now the second input. The PowerPoint template.

Here's the constraint — and this is a deliberate design decision — the template contains EXACTLY one slide. Not ten, not fifty. One.

Why? Because that one slide is the MASTER. It has the MSI branding, the table layout, the fonts, the colors — all of it. The tool's job is to clone it as many times as needed, then fill each clone with different data.

The diagram shows this as: "1 slide → Clone N times."

N here is simply the number of data rows in the Excel file. If you have 30 products, you get 30 clones. If you have 5, you get 5.

Now, how does cloning work in python-pptx? This is actually not a built-in feature of the library. There's no `slide.duplicate()` method. So we have to do it manually at the XML level.

Every slide in a PowerPoint file is really just an XML document. All the shapes — text boxes, images, tables — live inside an XML element called `_spTree`, the shape tree. To clone a slide, we:

One — add a blank new slide to the presentation.

Two — clear its shape tree.

Three — deep copy every XML element from the SOURCE slide's shape tree and paste it into the new one.

The keyword there is DEEP copy. Not a reference, not a shallow copy — a deep copy. Because if you did a shallow copy, all your cloned slides would be pointing to the same objects in memory. Edit one, you edit all of them. Deep copy gives each slide its own independent set of elements.

[PAUSE]

The Processing Loop — Where It All Comes Together

So now we have our `image_map`, we have our `DataFrame`, and we have `N` cloned slides in the presentation. This is where the main loop runs.

We iterate. For every row in the `DataFrame` — meaning for every product — we grab the corresponding slide. Slide zero gets row zero's data, slide one gets row one's data, and so on.

Step one inside the loop — Text Replacement.

We build a replacements dictionary for THIS specific row. It looks like: `"[ITEM_NUM]"` maps to `"SKU-1042"`, `"[RETAIL]"` maps to `"$299.00"`, `"[FEAT_1]"` maps to `"Stain resistant surface"`, and so on.

The template slide has these exact strings — `[ITEM_NUM]`, `[RETAIL]` — typed into text boxes as literal placeholder tags. The script finds them and swaps them out.

But here's the nuance. We don't just replace the whole paragraph's text object. We specifically target `paragraph.runs[0]` — the first run in each paragraph — and we clear all subsequent runs.

Why? Because PowerPoint stores formatting at the run level. Each run has its own font size, color, bold setting. If you nuke the whole paragraph and write fresh text, you lose the formatting. By writing into `runs[0]` only, the formatting survives. The text changes, the style doesn't. That's intentional preservation of the MSI branding.

[PAUSE]

Step two inside the loop — Image Insertion.

After text is handled, we look at our `image_map`. Remember, it's a dictionary keyed by Excel row number.

We calculate what Excel row number this product corresponds to. Then we ask: is that row number in the `image_map`?

If YES — there's a product image for this row. We insert it.

If NO — there's no image. We just clear the placeholder text so it doesn't say "Insert product picture here" on the final slide.

When we do insert an image, we can't just drop it in at its original dimensions. The placeholder box in the template has a specific aspect ratio — it might be roughly square, or wider than it is tall. The product image might be portrait, landscape, or square. They might not match.

So we use Pillow to letterbox the image. We calculate the target ratio of the placeholder box. If the image is wider than the box, we add white padding on the top and bottom. If it's taller, we pad the left and right. This way the image always fills the box exactly, never gets stretched, never gets cropped.

Then we save that padded image to a BytesIO object — which is just an in-memory file, no disk write — and we call `slide.shapes.add_picture()` at the exact same coordinates and dimensions as the original placeholder box.

The Output

Once the loop finishes — all N slides have been filled — we call `prs.save()` on the presentation object. But we don't save to a file path. We save to another BytesIO object. Then we extract the bytes from it and return them.

The caller — the button handler in the UI — takes those bytes, writes them to an actual file on disk, and shows the user a success message with download instructions.

The output is one clean .pptx file with N slides. Each slide is a filled, branded product submission.

Why This Architecture Makes Sense

Let me close with the "why."

Every design decision in this pipeline is about keeping things SEPARATED and COMPOSABLE.

The Excel file is responsible for data AND images — but those two things are extracted by different libraries into different data structures. They don't interfere with each other.

The PowerPoint template is responsible for visual layout — but it never touches data. The script handles the mapping.

The loop is thin. Each iteration does exactly two things: replace text, insert image. Nothing else.

And the whole thing runs without writing anything to disk until the very last step. Every intermediate result — image bytes, padded image, output presentation — lives in memory as

BytesIO objects. That's cleaner, faster, and works reliably in cloud environments like Google Colab where disk access can be unpredictable.

That's the pipeline. Top to bottom. Two files in, one file out, clean separation throughout.

Any questions?

[End of transcript]